```
// -------- file: src/main.ts --------
import { createSSRApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

export function createApp() {
  const app = createSSRApp(App)
  const pinia = createPinia()
  app.use(pinia)
  return {
    app,
    pinia
  }
}


// -------- file: src/App.vue --------
<script setup lang="ts">
import { onLaunch, onShow, onHide, onError } from '@dcloudio/uni-app'
import { useServerStore } from '@/stores/server'
import { useUserStore } from '@/stores/user'

onLaunch(() => {
  console.log('App Launch')
  // 恢复本地存储的配置
  const serverStore = useServerStore()
  const userStore = useUserStore()
  serverStore.restoreFromStorage()
  userStore.restore()

  // #ifdef APP-PLUS
  // 监听系统通话状态变化
  setupCallStateListener()
  // 设置Android返回键处理
  setupBackButtonHandler()
  // #endif
})

onShow(() => {
  console.log('App Show')
  // 检查是否有未完成的通话需要登记
  checkPendingCall()
})

onHide(() => {
  console.log('App Hide')
})

// 全局错误处理
onError((err) => {
```

```
  console.error('[App] 全局错误:', err)
})

// 检查是否有未完成的通话
const checkPendingCall = () => {
  const currentCall = uni.getStorageSync('currentCall')
  if (currentCall && currentCall.callId) {
    console.log('[App] 发现未完成的通话:', currentCall.callId)

    // 检查当前页面，避免重复跳转
    const pages = getCurrentPages()
    const currentPage = pages[pages.length - 1]
    const currentRoute = currentPage?.route || ''

    // 如果已经在通话结束页面，不重复跳转
    if (currentRoute.includes('call-ended')) {
      console.log('[App] 已在通话结束页面，跳过')
      return
    }

    // 计算通话时长
    const startTime = new Date(currentCall.startTime).getTime()
    const duration = Math.floor((Date.now() - startTime) / 1000)

    // 清除当前通话记录，避免重复弹出
    uni.removeStorageSync('currentCall')

    // 跳转到登记页面
    setTimeout(() => {
      uni.navigateTo({
        url: `/pages/call-ended/index?callId=${currentCall.callId}&name=${encodeURICompone
        fail: (err) => {
          console.error('[App] 跳转失败:', err)
          // 如果跳转失败，恢复通话记录
          uni.setStorageSync('currentCall', currentCall)
        }
      })
    }, 500)
  }
}

// #ifdef APP-PLUS
// 设置通话状态监听器
const setupCallStateListener = () => {
  // 监听应用从后台返回前台
  // @ts-expect-error - plus.globalEvent 在运行时存在
  plus.globalEvent.addEventListener('resume', () => {
    console.log('[App] 应用从后台返回')
    // 延迟检查，等待系统通话界面完全关闭
    setTimeout(() => {
```

```
      checkPendingCall()
    }, 1000)
  })
}

// 设置Android返回键处理（防止误触退出）
const setupBackButtonHandler = () => {
  let lastBackTime = 0
  plus.key.addEventListener('backbutton', () => {
    const pages = getCurrentPages()
    if (pages.length > 1) {
      // 有页面栈，正常返回
      uni.navigateBack({})
    } else {
      // 最后一页，双击退出
      const now = Date.now()
      if (now - lastBackTime < 2000) {
        plus.runtime.quit()
      } else {
        lastBackTime = now
        uni.showToast({
          title: '再按一次退出应用',
          icon: 'none',
          duration: 2000
        })
      }
    }
  })
}
// #endif
</script>

<style lang="scss">
/* 全局样式 */
page {
  background-color: #f5f5f5;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Ar
  width: 100%;
  overflow-x: hidden;
}

/* 禁止水平滚动 */
html, body, #app {
  width: 100%;
  max-width: 100%;
  overflow-x: hidden;
}

/* 通用类 */
.flex-row {
```

```
  display: flex;
  flex-direction: row;
}

.flex-col {
  display: flex;
  flex-direction: column;
}

.flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}

.flex-between {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.flex-1 {
  flex: 1;
}

/* 安全区域 */
.safe-area-bottom {
  padding-bottom: constant(safe-area-inset-bottom);
  padding-bottom: env(safe-area-inset-bottom);
}
</style>


// -------- file: src/config/app.ts --------
/**
 * APP 应用配置常量
 * 统一管理版本号、应用信息等，避免硬编码分散在各处
 */

/** 应用版本号 - 修改版本时只需改这一处 */
export const APP_VERSION = '1.0.0'

/** 应用版本码（用于安卓 versionCode，每次发版递增） */
export const APP_VERSION_CODE = 100

/** 应用名称 */
export const APP_NAME = 'CRM外呼助手'

/** 应用标语 */
export const APP_SLOGAN = '高效外呼 · 智能管理'
```

```
/** 应用包名（Android） */
export const APP_PACKAGE_NAME = 'com.xianhu.crm.dialer'

/** 公司名称 */
export const COMPANY_NAME = '广州仙狐网络科技有限公司'

/** 客服电话 */
export const SERVICE_PHONE = '13570727234'

/** 客服邮箱 */
export const SERVICE_EMAIL = 'xianhuquwang@163.com'

/** 客服微信 */
export const SERVICE_WECHAT = 'nxys789'


// -------- file: src/api/auth.ts --------
import { request } from '@/utils/request'
import type { UserInfo, DeviceInfo } from '@/stores/user'

// 登录响应
interface LoginResponse {
  token: string
  expiresIn: number
  user: UserInfo
}

// APP登录
export const login = (data: {
  username: string
  password: string
  deviceInfo?: Partial<DeviceInfo>
}): Promise<LoginResponse> => {
  return request({
    url: '/mobile/login',
    method: 'POST',
    data,
    showLoading: true,
    loadingText: '登录中...'
  })
}

// 绑定设备响应
interface BindResponse {
  deviceId: string
  userId: string
  wsToken: string
  wsUrl: string
}
```

```
// 扫码绑定设备
export const bindDevice = (data: {
  bindToken: string
  phoneNumber?: string
  deviceInfo: DeviceInfo
}): Promise<BindResponse> => {
  return request({
    url: '/mobile/bind',
    method: 'POST',
    data,
    showLoading: true,
    loadingText: '绑定中...'
  })
}

// 解绑设备
export const unbindDevice = (deviceId?: string): Promise<void> => {
  return request({
    url: '/mobile/unbind',
    method: 'DELETE',
    data: { deviceId },
    showLoading: true,
    loadingText: '解绑中...'
  })
}

// 获取设备状态
interface DeviceStatus {
  bound: boolean
  device: {
    id: number
    phoneNumber: string
    deviceId: string
    deviceName: string
    deviceModel: string
    osType: string
    osVersion: string
    appVersion: string
    onlineStatus: string
    lastActiveAt: string
    bindTime: string
  } | null
}

export const getDeviceStatus = (): Promise<DeviceStatus> => {
  return request({
    url: '/mobile/device/status',
    method: 'GET',
    showLoading: false
```

```
  })
}


// -------- file: src/api/call.ts --------
import { request, uploadFile } from '@/utils/request'

// 上报通话状态
export const reportCallStatus = (data: {
  callId: string
  status: 'dialing' | 'ringing' | 'connected' | 'ended' | 'missed' | 'rejected'
  timestamp?: string
}): Promise<void> => {
  return request({
    url: '/mobile/call/status',
    method: 'POST',
    data: {
      ...data,
      timestamp: data.timestamp || new Date().toISOString()
    },
    showLoading: false
  })
}

// 上报通话结束
export const reportCallEnd = (data: {
  callId: string
  status: 'connected' | 'missed' | 'rejected' | 'busy' | 'failed'
  startTime?: string
  endTime: string
  duration: number
  hasRecording: boolean
}): Promise<{ callId: string; recordingUploadUrl: string }> => {
  return request({
    url: '/mobile/call/end',
    method: 'POST',
    data,
    showLoading: false
  })
}

// 上传录音文件
export const uploadRecording = (callId: string, filePath: string): Promise<{
  recordingUrl: string
  fileSize: number
}> => {
  return uploadFile({
    url: '/mobile/recording/upload',
    filePath,
    name: 'file',
```

```
    formData: { callId }
  })
}

// 提交通话跟进记录
export const submitCallFollowup = (data: {
  callId: string
  notes?: string
  tags?: string[]
  intention?: 'high' | 'medium' | 'low' | 'none'
  followUpRequired?: boolean
  nextFollowUpDate?: string
  customerId?: string
}): Promise<{ callId: string; customerId: string; synced: boolean }> => {
  return request({
    url: '/mobile/call/followup',
    method: 'POST',
    data,
    showLoading: true,
    loadingText: '保存中...'
  })
}

// 通话记录
export interface CallRecord {
  id: string
  customerId?: string
  customerName?: string
  customerPhone: string
  callType: 'inbound' | 'outbound'
  callStatus: 'connected' | 'missed' | 'busy' | 'failed' | 'rejected'
  startTime: string
  duration: number
  hasRecording: boolean
  notes?: string
  callTags?: string[]
}

// 获取通话记录列表
export const getCallList = (params: {
  page?: number
  pageSize?: number
  callType?: 'inbound' | 'outbound'
  startDate?: string
  endDate?: string
}): Promise<{
  records: CallRecord[]
  total: number
  page: number
  pageSize: number
```

```
}> => {
  return request({
    url: '/mobile/calls',
    method: 'GET',
    data: params,
    showLoading: false
  })
}

// 通话详情
export interface CallDetail extends CallRecord {
  customerLevel?: string
  customerTags?: string[]
  followStatus?: string
  endTime?: string
  recordingUrl?: string
  followUpRequired?: boolean
  userId?: string
  userName?: string
  createdAt?: string
  followUpRecords?: Array<{
    id: string
    content: string
    intention?: string
    nextFollowUpDate?: string
    userName: string
    createdAt: string
  }>
}

// 获取通话详情
export const getCallDetail = (callId: string): Promise<CallDetail> => {
  return request({
    url: `/mobile/call/${callId}`,
    method: 'GET',
    showLoading: true
  })
}

// 今日统计
export interface TodayStats {
  totalCalls: number
  connectedCalls: number
  missedCalls: number
  inboundCalls: number
  outboundCalls: number
  totalDuration: number
  avgDuration: number
  connectRate: number
}
```

```
// 获取今日统计
export const getTodayStats = (): Promise<TodayStats> => {
  return request({
    url: '/mobile/stats/today',
    method: 'GET',
    showLoading: false
  })
}

// 获取统计（支持时间范围）
export const getStats = (period: 'today' | 'week' | 'month' = 'today'): Promise<TodayStats
  return request({
    url: '/mobile/stats',
    method: 'GET',
    data: { period },
    showLoading: false
  })
}


// -------- file: src/services/websocket.ts --------
/**
 * WebSocket 服务
 * 用于与服务器保持实时通信，接收拨号指令等
 */
import { useUserStore } from '@/stores/user'
import { useServerStore } from '@/stores/server'
import { callStateService } from './callStateService'
import { APP_VERSION } from '@/config/app'

// 消息类型
export interface WsMessage {
  type: string
  messageId?: string
  timestamp?: number
  data?: any
}

// 拨号请求
export interface DialRequest {
  callId: string
  phoneNumber: string
  customerName?: string
  customerId?: string
  source: string
  operatorId?: string
  operatorName?: string
}
```

```
class WebSocketService {
  private socket: UniApp.SocketTask | null = null
  private reconnectAttempts = 0
  private maxReconnectAttempts = 10
  private reconnectDelay = 3000
  private heartbeatTimer: number | null = null
  private heartbeatInterval = 30000 // 30秒心跳

  // 连接状态
  public isConnected = false

  // 事件回调
  private onDialRequestCallback: ((data: DialRequest) => void) | null = null
  private onDialCancelCallback: ((data: any) => void) | null = null
  private onDeviceUnbindCallback: (() => void) | null = null

  // 连接WebSocket
  connect() {
    const userStore = useUserStore()
    const serverStore = useServerStore()

    // 如果已经连接，不重复连接
    if (this.isConnected && this.socket) {
      console.log('[WebSocket] 已连接，跳过重复连接')
      return
    }

    if (!userStore.wsToken) {
      console.log('[WebSocket] 缺少 wsToken，请重新扫码绑定')
      uni.$emit('ws:need_rebind', { reason: 'missing_token' })
      return
    }

    // 优先使用 userStore 中保存的 wsUrl，否则从 serverStore 计算
    const baseWsUrl = userStore.wsUrl || serverStore.wsUrl
    if (!baseWsUrl) {
      console.log('[WebSocket] 缺少 WebSocket 地址，请重新扫码绑定')
      uni.$emit('ws:need_rebind', { reason: 'missing_url' })
      return
    }

    if (this.socket) {
      console.log('[WebSocket] 已有连接，先关闭')
      this.disconnect()
    }

    // 构建完整的 WebSocket URL
    // 如果 baseWsUrl 已经包含 /ws/mobile，则直接使用
    let wsUrl = baseWsUrl
    if (!wsUrl.includes('/ws/mobile')) {
```

```
    wsUrl = `${baseWsUrl}/ws/mobile`
  }

  // 确保使用正确的协议
  if (wsUrl.startsWith('http://')) {
    wsUrl = wsUrl.replace('http://', 'ws://')
  } else if (wsUrl.startsWith('https://')) {
    wsUrl = wsUrl.replace('https://', 'wss://')
  }

  // 🔥 修复：确保 wsUrl 不包含 /api/v1 前缀
  wsUrl = wsUrl.replace('/api/v1/ws/mobile', '/ws/mobile')
  wsUrl = wsUrl.replace('/api/ws/mobile', '/ws/mobile')

  wsUrl = `${wsUrl}?token=${userStore.wsToken}`

  console.log('[WebSocket] 正在连接:', wsUrl)
  console.log('[WebSocket] wsToken长度:', userStore.wsToken.length)
  console.log('[WebSocket] 当前wsUrl:', userStore.wsUrl)
  console.log('[WebSocket] serverStore.wsUrl:', serverStore.wsUrl)

  try {
    this.socket = uni.connectSocket({
      url: wsUrl,
      success: () => {
        console.log('[WebSocket] 连接请求已发送')
      },
      fail: (err) => {
        console.error('[WebSocket] 连接请求失败:', JSON.stringify(err))
        // 连接失败时触发重连
        this.scheduleReconnect()
      }
    })

    this.setupListeners()
  } catch (e) {
    console.error('[WebSocket] 创建连接异常:', e)
    this.scheduleReconnect()
  }
}

// 设置监听器
private setupListeners() {
  if (!this.socket) return

  // 连接成功
  this.socket.onOpen(() => {
    console.log('[WebSocket] 连接成功')
    this.isConnected = true
    this.reconnectAttempts = 0
```

```
    // 发送设备上线消息
    this.sendDeviceOnline()

    // 启动心跳
    this.startHeartbeat()

    // 通知UI更新
    uni.$emit('ws:connected')
  })

  // 收到消息
  this.socket.onMessage((res) => {
    try {
      const message: WsMessage = JSON.parse(res.data as string)
      this.handleMessage(message)
    } catch (e) {
      console.error('[WebSocket] 解析消息失败:', e)
    }
  })

  // 连接关闭
  this.socket.onClose((res) => {
    console.log('[WebSocket] 连接关闭:', res)
    this.isConnected = false
    this.stopHeartbeat()

    uni.$emit('ws:disconnected')

    // 自动重连
    this.scheduleReconnect()
  })

  // 连接错误
  this.socket.onError((err) => {
    // 打印更详细的错误信息
    console.error('[WebSocket] 连接错误:', JSON.stringify(err))
    console.error('[WebSocket] 当前连接URL:', this.getCurrentWsUrl())
    this.isConnected = false
    uni.$emit('ws:error', err)
  })
}

// 处理消息
private handleMessage(message: WsMessage) {
  console.log('[WebSocket] 收到消息:', message.type, message.data)

  switch (message.type) {
    case 'DIAL_REQUEST':
    case 'DIAL_COMMAND':
```

```
    // 收到拨号指令 - 直接执行拨号，不弹窗确认
    if (this.onDialRequestCallback && message.data) {
      this.onDialRequestCallback(message.data)
    }
    // 直接执行拨号
    this.executeDial(message.data)
    break

  case 'DIAL_CANCEL':
    // 取消拨号
    if (this.onDialCancelCallback && message.data) {
      this.onDialCancelCallback(message.data)
    }
    break

  case 'CALL_END':
  case 'END_CALL':
    // 服务器发来的结束通话指令（CRM端结束通话）
    // 注意：系统级通话无法真正挂断，只能标记状态并弹出跟进页面
    console.log('[WebSocket] 收到服务器结束通话指令:', message.data)
    uni.$emit('call:end', message.data)
    uni.$emit('ws:call_ended', message.data)

    // 检查是否有当前通话，如果有则跳转到跟进页面
    const currentCall = uni.getStorageSync('currentCall')
    if (currentCall && currentCall.callId === message.data?.callId) {
      const startTime = new Date(currentCall.startTime).getTime()
      const duration = Math.floor((Date.now() - startTime) / 1000)

      // 清除当前通话记录
      uni.removeStorageSync('currentCall')

      // 显示提示
      uni.showModal({
        title: '通话已结束',
        content: 'CRM系统已标记通话结束，请填写跟进记录',
        showCancel: false,
        confirmText: '去填写',
        success: () => {
          uni.navigateTo({
            url: `/pages/call-ended/index?callId=${currentCall.callId}&name=${encodeUR
          })
        }
      })
    }
    break

  case 'DEVICE_UNBIND':
    // 设备被解绑
    if (this.onDeviceUnbindCallback) {
```

```
        this.onDeviceUnbindCallback()
      }
      this.handleDeviceUnbind()
      break

    case 'HEARTBEAT_ACK':
    case 'pong':
      // 心跳响应，不需要处理
      break

    default:
      console.log('[WebSocket] 未知消息类型:', message.type)
  }
}

// 执行拨号 - 直接调用系统拨号，使用callStateService监听通话状态
private executeDial(data: DialRequest) {
  if (!data || !data.phoneNumber) {
    console.error('[WebSocket] 拨号数据无效:', data)
    return
  }

  console.log('[WebSocket] 执行拨号:', data.phoneNumber, '客户:', data.customerName)

  // 保存当前通话信息到全局，供通话结束后使用
  const callStartTime = new Date().toISOString()
  uni.setStorageSync('currentCall', {
    callId: data.callId,
    phoneNumber: data.phoneNumber,
    customerName: data.customerName || '未知客户',
    customerId: data.customerId || '',
    startTime: callStartTime
  })

  // 上报拨号状态
  this.reportCallStatus(data.callId, 'dialing')

  // 启动通话状态监听服务
  callStateService.startMonitoring({
    callId: data.callId,
    phoneNumber: data.phoneNumber,
    customerName: data.customerName,
    customerId: data.customerId
  })

  // 设置状态变化回调
  callStateService.onStateChange((state, callInfo) => {
    console.log('[WebSocket] 通话状态变化:', state, callInfo)

    if (state === 'offhook') {
```

```
      // 通话已接通
      uni.showToast({
        title: '通话已接通',
        icon: 'success',
        duration: 1500
      })
    } else if (state === 'ringing') {
      // 对方响铃中
      uni.showToast({
        title: '对方响铃中...',
        icon: 'none',
        duration: 1500
      })
    }
  })

  // 设置通话结束回调
  callStateService.onCallEnd((callInfo, duration) => {
    console.log('[WebSocket] 通话结束回调:', callInfo, duration)
    // 清除当前通话记录
    uni.removeStorageSync('currentCall')
  })

  // 直接调用系统拨号
  // #ifdef APP-PLUS
  plus.device.dial(data.phoneNumber, false)
  console.log('[WebSocket] 系统拨号已发起')

  // 显示提示
  uni.showToast({
    title: '正在拨号...',
    icon: 'none',
    duration: 2000
  })
  // #endif

  // #ifndef APP-PLUS
  uni.makePhoneCall({
    phoneNumber: data.phoneNumber,
    success: () => {
      console.log('[WebSocket] 系统拨号成功')
    },
    fail: (err) => {
      console.error('[WebSocket] 系统拨号失败:', err)
      uni.showToast({ title: '拨号失败', icon: 'none' })
      callStateService.stopMonitoring()
    }
  })
  // #endif
}
```

```
// 处理设备解绑
private handleDeviceUnbind() {
  const userStore = useUserStore()
  userStore.clearDeviceInfo()

  uni.showModal({
    title: '设备已解绑',
    content: '您的设备已被管理员解绑，请重新绑定',
    showCancel: false,
    success: () => {
      uni.reLaunch({ url: '/pages/index/index' })
    }
  })
}

// 发送消息
send(type: string, data?: any) {
  if (!this.socket || !this.isConnected) {
    console.warn('[WebSocket] 未连接，无法发送消息')
    return
  }

  const message: WsMessage = {
    type,
    messageId: `msg_${Date.now()}`,
    timestamp: Date.now(),
    data
  }

  this.socket.send({
    data: JSON.stringify(message),
    fail: (err) => {
      console.error('[WebSocket] 发送消息失败:', err)
    }
  })
}

// 发送设备上线消息
private sendDeviceOnline() {
  const userStore = useUserStore()
  this.send('DEVICE_ONLINE', {
    deviceId: userStore.deviceInfo?.deviceId,
    appVersion: APP_VERSION
  })
}

// 上报通话状态
reportCallStatus(callId: string, status: string, extra?: any) {
  this.send('CALL_STATUS', {
```

```
    callId,
    status,
    timestamp: new Date().toISOString(),
    ...extra
  })
}

// 上报通话结束
reportCallEnd(callId: string, data: any) {
  this.send('CALL_ENDED', {
    callId,
    ...data
  })
}

// 启动心跳
private startHeartbeat() {
  this.stopHeartbeat()

  this.heartbeatTimer = setInterval(() => {
    if (this.isConnected) {
      this.send('HEARTBEAT')
    }
  }, this.heartbeatInterval) as unknown as number
}

// 停止心跳
private stopHeartbeat() {
  if (this.heartbeatTimer) {
    clearInterval(this.heartbeatTimer)
    this.heartbeatTimer = null
  }
}

// 重连调度
private scheduleReconnect() {
  if (this.reconnectAttempts >= this.maxReconnectAttempts) {
    console.error('[WebSocket] 达到最大重连次数')
    uni.$emit('ws:max_reconnect')
    return
  }

  this.reconnectAttempts++
  const delay = this.reconnectDelay * Math.pow(1.5, this.reconnectAttempts - 1)

  console.log(`[WebSocket] ${delay}ms后重连，第${this.reconnectAttempts}次`)

  setTimeout(() => {
    this.connect()
  }, Math.min(delay, 30000)) // 最大30秒
```

```
  }

  // 断开连接
  disconnect() {
    this.stopHeartbeat()

    if (this.socket) {
      this.socket.close({})
      this.socket = null
    }

    this.isConnected = false
  }

  // 设置拨号请求回调
  onDialRequest(callback: (data: DialRequest) => void) {
    this.onDialRequestCallback = callback
  }

  // 设置取消拨号回调
  onDialCancel(callback: (data: any) => void) {
    this.onDialCancelCallback = callback
  }

  // 设置设备解绑回调
  onDeviceUnbind(callback: () => void) {
    this.onDeviceUnbindCallback = callback
  }

  // 获取当前WebSocket URL（用于调试）
  private getCurrentWsUrl(): string {
    const userStore = useUserStore()
    const serverStore = useServerStore()
    const baseWsUrl = userStore.wsUrl || serverStore.wsUrl
    if (!baseWsUrl) return '(无wsUrl)'
    let wsUrl = baseWsUrl
    if (!wsUrl.includes('/ws/mobile')) {
      wsUrl = `${baseWsUrl}/ws/mobile`
    }
    return wsUrl
  }
}

// 导出单例
export const wsService = new WebSocketService()
export default wsService


// -------- file: src/services/callStateService.ts --------
/**
```

```
 * 通话状态监听服务
 *
 * 用于监听系统通话状态变化，实现：
 * 1. 检测通话接通 - 开始计时
 * 2. 检测通话挂断 - 自动结束并上传录音
 * 3. 自动扫描系统通话录音并上传
 */

import { wsService } from './websocket'
import { reportCallEnd } from '@/api/call'
import { recordingService } from './recordingService'

// 通话状态
export type CallState = 'idle' | 'dialing' | 'ringing' | 'offhook' | 'ended'

// 当前通话信息
interface CurrentCallInfo {
  callId: string
  phoneNumber: string
  customerName?: string
  customerId?: string
  startTime: number
  connectTime?: number
  state: CallState
  recordingPath?: string
}

class CallStateService {
  private currentCall: CurrentCallInfo | null = null
  private stateCheckTimer: number | null = null
  private lastPhoneState: number = 0 // 0=IDLE, 1=RINGING, 2=OFFHOOK
  private isMonitoring = false

  // 回调函数
  private onStateChangeCallback: ((state: CallState, callInfo: CurrentCallInfo | null) =>
  private onCallEndCallback: ((callInfo: CurrentCallInfo, duration: number) => void) | nul

  /**
   * 开始监听通话状态
   */
  startMonitoring(callInfo: {
    callId: string
    phoneNumber: string
    customerName?: string
    customerId?: string
  }) {
    console.log('[CallStateService] 开始监听通话状态:', callInfo)

    this.currentCall = {
      ...callInfo,
```

```
    startTime: Date.now(),
    state: 'dialing'
  }

  this.isMonitoring = true
  this.lastPhoneState = 0

  // 启动状态检测
  this.startStateCheck()
}

/**
 * 停止监听
 */
stopMonitoring() {
  console.log('[CallStateService] 停止监听')
  this.isMonitoring = false
  this.stopStateCheck()
  this.currentCall = null
}

/**
 * 启动状态检测定时器
 */
private startStateCheck() {
  this.stopStateCheck()

  // 每500ms检测一次通话状态
  this.stateCheckTimer = setInterval(() => {
    this.checkPhoneState()
  }, 500) as unknown as number
}

/**
 * 停止状态检测
 */
private stopStateCheck() {
  if (this.stateCheckTimer) {
    clearInterval(this.stateCheckTimer)
    this.stateCheckTimer = null
  }
}

/**
 * 检测系统通话状态
 */
private checkPhoneState() {
  if (!this.isMonitoring || !this.currentCall) return

  // #ifdef APP-PLUS
```

```
  try {
    // 使用 Android 原生 API 获取通话状态
    const main = plus.android.runtimeMainActivity()
    const Context = plus.android.importClass('android.content.Context')
    plus.android.importClass('android.telephony.TelephonyManager')

    const telephonyManager = (main as any).getSystemService((Context as any).TELEPHONY_S
    const callState = telephonyManager.getCallState()

    // callState: 0=IDLE, 1=RINGING, 2=OFFHOOK
    if (callState !== this.lastPhoneState) {
      console.log('[CallStateService] 通话状态变化:', this.lastPhoneState, '->', callSta
      this.handleStateChange(callState)
      this.lastPhoneState = callState
    }
  } catch (e) {
    console.error('[CallStateService] 获取通话状态失败:', e)
  }
  // #endif
}

/**
 * 处理状态变化
 */
private handleStateChange(newState: number) {
  if (!this.currentCall) return

  const oldState = this.currentCall.state

  switch (newState) {
    case 0: // IDLE - 通话结束
      if (oldState !== 'idle' && oldState !== 'ended') {
        this.currentCall.state = 'ended'
        this.onCallEnded()
      }
      break

    case 1: // RINGING - 响铃中
      if (oldState === 'dialing') {
        this.currentCall.state = 'ringing'
        this.notifyStateChange('ringing')
      }
      break

    case 2: // OFFHOOK - 通话中（接通）
      if (oldState !== 'offhook') {
        this.currentCall.state = 'offhook'
        this.currentCall.connectTime = Date.now()
        this.notifyStateChange('offhook')
```

```
        // 通知WebSocket通话已接通
        wsService.reportCallStatus(this.currentCall.callId, 'connected')
      }
      break
  }
}

/**
 * 通话结束处理
 */
private async onCallEnded() {
  if (!this.currentCall) return

  console.log('[CallStateService] 通话结束')

  // 计算通话时长
  let duration = 0
  if (this.currentCall.connectTime) {
    duration = Math.floor((Date.now() - this.currentCall.connectTime) / 1000)
  }

  const callInfo = { ...this.currentCall }
  const endTime = Date.now()

  // 通知状态变化
  this.notifyStateChange('ended')

  // 先通过WebSocket通知服务器通话结束（不等待录音）
  wsService.reportCallEnd(callInfo.callId, {
    status: duration > 0 ? 'connected' : 'missed',
    startTime: new Date(callInfo.startTime).toISOString(),
    endTime: new Date(endTime).toISOString(),
    duration,
    hasRecording: false, // 先标记为无录音，后续找到再更新
    endReason: 'system_hangup',
  })

  // 上报通话结束到API
  try {
    await reportCallEnd({
      callId: callInfo.callId,
      status: duration > 0 ? 'connected' : 'missed',
      startTime: new Date(callInfo.startTime).toISOString(),
      endTime: new Date(endTime).toISOString(),
      duration,
      hasRecording: false,
    })
  } catch (e) {
    console.error('[CallStateService] 上报通话结束失败:', e)
  }
```

```
  // 触发回调
  if (this.onCallEndCallback) {
    this.onCallEndCallback(callInfo, duration)
  }

  // 保存通话信息供跟进页面使用
  uni.setStorageSync('lastEndedCall', {
    callId: callInfo.callId,
    customerName: callInfo.customerName,
    customerId: callInfo.customerId,
    phoneNumber: callInfo.phoneNumber,
    duration,
    endTime: new Date(endTime).toISOString(),
    wasConnected: duration > 0,
    hasRecording: false, // 先标记为无录音
  })

  // 停止监听
  this.stopMonitoring()

  // 跳转到通话结束页面
  setTimeout(() => {
    uni.navigateTo({
      url: `/pages/call-ended/index?callId=${callInfo.callId}&name=${encodeURIComponent(
    })
  }, 300)

  // 异步处理录音（在后台进行，不阻塞用户操作）
  this.processRecordingAsync(callInfo, duration, endTime)
}

/**
 * 异步处理录音文件
 */
private async processRecordingAsync(
  callInfo: CurrentCallInfo,
  duration: number,
  endTime: number
) {
  if (duration <= 0) {
    console.log('[CallStateService] 通话未接通，跳过录音处理')
    return
  }

  // 检查是否开启了自动上传
  try {
    const callSettings = uni.getStorageSync('callSettings')
    if (callSettings) {
      const settings = JSON.parse(callSettings)
```

```
    if (!settings.autoUploadRecording) {
      console.log('[CallStateService] 自动上传已关闭，跳过录音处理')
      return
    }
  }
} catch (e) {
  console.error('[CallStateService] 读取设置失败:', e)
}

console.log('[CallStateService] 开始异步处理录音...')

try {
  // 使用录音服务查找并上传录音
  const result = await recordingService.processCallRecording({
    callId: callInfo.callId,
    phoneNumber: callInfo.phoneNumber,
    startTime: callInfo.startTime,
    endTime: endTime,
    duration: duration,
  })

  if (result.found && result.uploaded) {
    console.log('[CallStateService] 录音处理成功:', result.recordingPath)

    // 通知服务器录音已上传
    wsService.reportCallEnd(callInfo.callId, {
      status: 'connected',
      startTime: new Date(callInfo.startTime).toISOString(),
      endTime: new Date(endTime).toISOString(),
      duration,
      hasRecording: true,
      recordingPath: result.recordingPath,
      endReason: 'recording_uploaded',
    })

    // 显示成功提示
    uni.showToast({
      title: '录音已上传',
      icon: 'success',
      duration: 2000,
    })
  } else if (result.found) {
    console.log('[CallStateService] 找到录音但上传失败')
    uni.showToast({
      title: '录音上传失败',
      icon: 'none',
      duration: 2000,
    })
  } else {
    console.log('[CallStateService] 未找到录音文件')
```

```
      // 不显示提示，可能是用户没有开启系统录音
    }
  } catch (e) {
    console.error('[CallStateService] 录音处理失败:', e)
  }
}

/**
 * 通知状态变化
 */
private notifyStateChange(state: CallState) {
  if (this.onStateChangeCallback && this.currentCall) {
    this.onStateChangeCallback(state, this.currentCall)
  }
}

/**
 * 设置状态变化回调
 */
onStateChange(callback: (state: CallState, callInfo: CurrentCallInfo | null) => void) {
  this.onStateChangeCallback = callback
}

/**
 * 设置通话结束回调
 */
onCallEnd(callback: (callInfo: CurrentCallInfo, duration: number) => void) {
  this.onCallEndCallback = callback
}

/**
 * 获取当前通话信息
 */
getCurrentCall(): CurrentCallInfo | null {
  return this.currentCall
}

/**
 * 获取当前通话时长（秒）
 */
getCurrentDuration(): number {
  if (!this.currentCall || !this.currentCall.connectTime) return 0
  return Math.floor((Date.now() - this.currentCall.connectTime) / 1000)
}

/**
 * 是否正在通话中
 */
isInCall(): boolean {
  return this.isMonitoring && this.currentCall !== null &&
```

```
          (this.currentCall.state === 'offhook' || this.currentCall.state === 'ringing' |
  }
}

// 导出单例
export const callStateService = new CallStateService()
export default callStateService


// -------- file: src/services/recordingService.ts --------
/**
 * 通话录音服务
 *
 * 功能：
 * 1. 扫描系统通话录音文件夹
 * 2. 匹配通话时间找到对应录音
 * 3. 上传录音文件到服务器
 *
 * 支持的录音文件夹路径（不同手机品牌）：
 * - 小米: /storage/emulated/0/MIUI/sound_recorder/call_rec/
 * - 华为: /storage/emulated/0/Sounds/CallRecord/
 * - OPPO: /storage/emulated/0/Recordings/Call/
 * - VIVO: /storage/emulated/0/Record/Call/
 * - 三星: /storage/emulated/0/Call/
 * - 通用: /storage/emulated/0/Recordings/
 */

import { uploadRecording } from '@/api/call'

// 常见手机品牌的录音文件夹路径
const RECORDING_PATHS = [
  // 小米
  '/storage/emulated/0/MIUI/sound_recorder/call_rec/',
  '/storage/emulated/0/MIUI/sound_recorder/',
  // 华为
  '/storage/emulated/0/Sounds/CallRecord/',
  '/storage/emulated/0/record/',
  '/storage/emulated/0/Record/',
  // OPPO
  '/storage/emulated/0/Recordings/Call/',
  '/storage/emulated/0/Recordings/',
  // VIVO
  '/storage/emulated/0/Record/Call/',
  '/storage/emulated/0/Record/',
  // 三星
  '/storage/emulated/0/Call/',
  '/storage/emulated/0/Recordings/Call recordings/',
  // 一加
  '/storage/emulated/0/Record/PhoneRecord/',
  // 通用路径
```

```
  '/storage/emulated/0/Recordings/',
  '/storage/emulated/0/AudioRecorder/',
  '/storage/emulated/0/CallRecordings/',
  '/sdcard/MIUI/sound_recorder/call_rec/',
  '/sdcard/Recordings/',
]

// 录音文件扩展名
const AUDIO_EXTENSIONS = ['.mp3', '.amr', '.wav', '.m4a', '.3gp', '.aac', '.ogg']

interface RecordingFile {
  path: string
  name: string
  size: number
  lastModified: number
}

interface CallInfo {
  callId: string
  phoneNumber: string
  startTime: number // 通话开始时间戳
  endTime: number // 通话结束时间戳
  duration: number // 通话时长（秒）
}

class RecordingService {
  private isScanning = false
  private lastScanTime = 0
  private knownRecordings: Set<string> = new Set()

  /**
   * 检查存储权限（适配 Android 11+ Scoped Storage）
   */
  async checkPermissions(): Promise<boolean> {
    // #ifdef APP-PLUS
    return new Promise((resolve) => {
      try {
        // 检查 Android 版本
        const Build = plus.android.importClass('android.os.Build')
        const sdkVersion = (Build as any).VERSION.SDK_INT

        if (sdkVersion >= 30) {
          // Android 11+: 检查 MANAGE_EXTERNAL_STORAGE 权限
          const Environment = plus.android.importClass('android.os.Environment')
          if ((Environment as any).isExternalStorageManager()) {
            resolve(true)
            return
          }
          // 请求所有文件访问权限
          const Settings = plus.android.importClass('android.provider.Settings')
```

```
    const Intent = plus.android.importClass('android.content.Intent')
    const Uri = plus.android.importClass('android.net.Uri')
    const main = plus.android.runtimeMainActivity()
    const packageName = (main as any).getPackageName()

    uni.showModal({
      title: '需要存储权限',
      content: '为了自动扫描和上传通话录音，需要授予"所有文件访问"权限。',
      confirmText: '去授权',
      success: (res) => {
        if (res.confirm) {
          try {
            const intent = new (Intent as any)((Settings as any).ACTION_MANAGE_APP_A
            intent.setData((Uri as any).parse('package:' + packageName))
            ;(main as any).startActivity(intent)
          } catch (_e) {
            // 降级方案
            try {
              const intent2 = new (Intent as any)((Settings as any).ACTION_MANAGE_AL
              ;(main as any).startActivity(intent2)
            } catch (_e2) {
              console.error('[RecordingService] 无法打开文件访问权限设置')
            }
          }
        }
        // 延迟检查权限结果
        setTimeout(() => {
          resolve((Environment as any).isExternalStorageManager())
        }, 1000)
      }
    })
  } else if (sdkVersion >= 33) {
    // Android 13+: 使用 READ_MEDIA_AUDIO
    plus.android.requestPermissions(
      ['android.permission.READ_MEDIA_AUDIO'],
      (result: any) => {
        resolve(result.granted && result.granted.length >= 1)
      },
      (_error: any) => {
        resolve(false)
      }
    )
  } else {
    // Android 10 及以下: 传统存储权限
    plus.android.requestPermissions(
      [
        'android.permission.READ_EXTERNAL_STORAGE',
        'android.permission.WRITE_EXTERNAL_STORAGE',
      ],
      (result: any) => {
```

```
            console.log('[RecordingService] 权限请求结果:', result)
            const granted = result.granted && result.granted.length >= 2
            resolve(granted)
          },
          (error: any) => {
            console.error('[RecordingService] 权限请求失败:', error)
            resolve(false)
          }
        )
      }
    } catch (e) {
      console.error('[RecordingService] 权限检查异常:', e)
      // 降级到旧方案
      plus.android.requestPermissions(
        [
          'android.permission.READ_EXTERNAL_STORAGE',
          'android.permission.WRITE_EXTERNAL_STORAGE',
        ],
        (result: any) => {
          resolve(result.granted && result.granted.length >= 2)
        },
        (_error: any) => {
          resolve(false)
        }
      )
    }
  })
  // #endif

  // #ifndef APP-PLUS
  return false
  // #endif
}

/**
 * 获取设备品牌
 */
getDeviceBrand(): string {
  // #ifdef APP-PLUS
  try {
    const Build = plus.android.importClass('android.os.Build')
    const brand = (Build as any).BRAND || ''
    console.log('[RecordingService] 设备品牌:', brand)
    return brand.toLowerCase()
  } catch (e) {
    console.error('[RecordingService] 获取设备品牌失败:', e)
  }
  // #endif
  return ''
}
```

```
    uni.hideLoading()
    console.error('打开录音设置失败:', e)
    uni.showToast({ title: '打开设置失败', icon: 'none' })
  }
  // #endif

  // #ifndef APP-PLUS
  // 非 APP 环境显示提示
  uni.showModal({
    title: '提示',
    content: '此功能需要在手机APP中使用。\n\n请在真机上安装APP后，进入设置页面开启系统录音
    showCancel: false,
    confirmText: '我知道了'
  })
  // #endif
}

// 跳转到关于页面
const goToAbout = () => {
  uni.navigateTo({ url: '/pages/about/index' })
}

// 解绑设备
const handleUnbind = () => {
  uni.showModal({
    title: '确认解绑',
    content: '解绑后将无法接收PC端拨号指令，确定要解绑吗？',
    success: async (res) => {
      if (res.confirm) {
        uni.showLoading({ title: '解绑中...' })
        try {
          wsService.disconnect()
          await unbindDevice(userStore.deviceInfo?.deviceId)
          userStore.clearDeviceInfo()
          uni.hideLoading()
          uni.showToast({ title: '解绑成功', icon: 'success' })
        } catch (e: any) {
          uni.hideLoading()
          console.error('解绑失败:', e)
          userStore.clearDeviceInfo()
          uni.showToast({ title: '已解绑', icon: 'success' })
        }
      }
    }
  })
}

// 退出登录
const handleLogout = () => {
  uni.showModal({
```

```
    title: '确认退出',
    content: '确定要退出登录吗？',
    success: (res) => {
      if (res.confirm) {
        wsService.send('DEVICE_OFFLINE', {
          deviceId: userStore.deviceInfo?.deviceId,
          reason: 'logout'
        })
        wsService.disconnect()
        uni.removeStorageSync('currentCall')
        // 注意：不清除 callSettings 和 uploadPasswordData
        // 这样管理员设置的自动上传和密码会保留
        // 防止员工通过退出登录来绑过密码保护
        userStore.logout()
        uni.reLaunch({ url: '/pages/login/index' })
      }
    }
  })
}
</script>

<style lang="scss" scoped>
.settings-page {
  min-height: 100vh;
  background: #f5f5f5;
  padding-bottom: 180rpx;
  width: 100%;
  overflow-x: hidden;
  box-sizing: border-box;

  .user-section {
    display: flex;
    align-items: center;
    background: #fff;
    padding: 40rpx 32rpx;
    margin-bottom: 24rpx;

    .avatar {
      width: 100rpx;
      height: 100rpx;
      background: linear-gradient(135deg, #6EE7B7 0%, #34D399 100%);
      border-radius: 50%;
      display: flex;
      align-items: center;
      justify-content: center;
      margin-right: 24rpx;

      text {
        font-size: 44rpx;
        color: #fff;
```

```
      font-weight: bold;
    }
  }

  .info {
    .name {
      font-size: 36rpx;
      font-weight: 600;
      color: #1F2937;
      display: block;
    }

    .dept {
      font-size: 26rpx;
      color: #6B7280;
      margin-top: 8rpx;
      display: block;
    }
  }
}

.section {
  margin-bottom: 24rpx;

  .section-title {
    font-size: 26rpx;
    color: #6B7280;
    padding: 0 32rpx;
    margin-bottom: 12rpx;
    display: block;
  }
}

.setting-group {
  background: #fff;

  .setting-item {
    display: flex;
    align-items: center;
    padding: 28rpx 32rpx;
    border-bottom: 1rpx solid #F3F4F6;

    &:last-child {
      border-bottom: none;
    }

    &.clickable {
      &:active {
        background: #F9FAFB;
      }
```

```
}

.label {
  font-size: 30rpx;
  color: #1F2937;
  flex: 1;
}

.value {
  font-size: 28rpx;
  color: #6B7280;

  &.bound, &.connected, &.enabled {
    color: #10B981;
  }

  &.disconnected {
    color: #EF4444;
  }

  &.unbound, &.disabled {
    color: #F59E0B;
  }
}

.arrow {
  font-size: 32rpx;
  color: #D1D5DB;
  margin-left: 12rpx;
}

.picker-value {
  color: #3B82F6;
  text-decoration: underline;
}

.connection-status, .recording-status {
  display: flex;
  align-items: center;

  .status-dot {
    width: 16rpx;
    height: 16rpx;
    border-radius: 50%;
    margin-right: 12rpx;

    &.connected, &.enabled {
      background: #10B981;
    }
```

```
        &.disconnected, &.disabled {
          background: #F59E0B;
        }

        &.unbound {
          background: #9CA3AF;
        }
      }

      .btn-mini-reconnect {
        margin-left: 16rpx;
        padding: 8rpx 20rpx;
        font-size: 24rpx;
        color: #34D399;
        background: rgba(52, 211, 153, 0.1);
        border: 1rpx solid #34D399;
        border-radius: 24rpx;
        line-height: 1.2;
      }

      .btn-mini-refresh {
        margin-left: 16rpx;
        padding: 8rpx 20rpx;
        font-size: 24rpx;
        color: #3B82F6;
        background: rgba(59, 130, 246, 0.1);
        border: 1rpx solid #3B82F6;
        border-radius: 24rpx;
        line-height: 1.2;
      }

      .checking-indicator {
        text {
          font-size: 26rpx;
          color: #6B7280;
        }
      }
    }

    .upload-status {
      display: flex;
      align-items: center;

      .lock-icon {
        font-size: 28rpx;
        margin-right: 12rpx;
      }
    }
  }
}
```

```
  .setting-tip {
    padding: 16rpx 32rpx;

    text {
      font-size: 24rpx;
      color: #6B7280;
      line-height: 1.6;
    }
  }

  .actions {
    padding: 48rpx 32rpx;

    button {
      width: 100%;
      height: 88rpx;
      border-radius: 16rpx;
      font-size: 30rpx;
      margin-bottom: 24rpx;
      border: none;

      &.btn-unbind {
        background: #fff;
        color: #F59E0B;
        border: 2rpx solid #F59E0B;
      }

      &.btn-logout {
        background: #fff;
        color: #EF4444;
        border: 2rpx solid #EF4444;
      }
    }
  }
}

// 密码弹窗样式
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}
```

```
.modal-content {
  width: 85%;
  max-width: 600rpx;
  background: #fff;
  border-radius: 24rpx;
  padding: 40rpx 32rpx;

  .modal-title {
    font-size: 36rpx;
    font-weight: 600;
    color: #1F2937;
    text-align: center;
    display: block;
    margin-bottom: 24rpx;
  }

  .modal-desc {
    font-size: 28rpx;
    color: #6B7280;
    display: block;
    margin-bottom: 20rpx;
  }

  .password-input-group {
    margin-bottom: 20rpx;

    .password-input {
      width: 100%;
      height: 88rpx;
      background: #F3F4F6;
      border-radius: 12rpx;
      padding: 0 24rpx;
      font-size: 32rpx;
      text-align: center;
      letter-spacing: 16rpx;
      box-sizing: border-box;
    }
  }

  .security-question {
    margin-bottom: 24rpx;

    .question-label {
      font-size: 28rpx;
      color: #1F2937;
      display: block;
      margin-bottom: 12rpx;
    }
```

```
    .answer-input {
      width: 100%;
      height: 88rpx;
      background: #F3F4F6;
      border-radius: 12rpx;
      padding: 0 24rpx;
      font-size: 28rpx;
      box-sizing: border-box;
    }
  }

  .forgot-password {
    text-align: right;
    margin-bottom: 24rpx;

    text {
      font-size: 26rpx;
      color: #3B82F6;
    }
  }

  .modal-buttons {
    display: flex;
    gap: 24rpx;
    margin-top: 32rpx;

    button {
      flex: 1;
      height: 88rpx;
      border-radius: 16rpx;
      font-size: 30rpx;
      border: none;

      &.btn-cancel {
        background: #F3F4F6;
        color: #6B7280;
      }

      &.btn-confirm {
        background: linear-gradient(135deg, #6EE7B7 0%, #34D399 100%);
        color: #fff;
      }
    }
  }
}
</style>


// -------- file: src/pages/about/index.vue --------
<template>
```

```
  <view class="about-page">
    <!-- 应用信息 -->
    <view class="app-info">
      <image class="app-logo" src="/static/logo.png" mode="aspectFit" />
      <text class="app-name">CRM外呼助手</text>
      <text class="app-version">版本 v{{ appVersion }}</text>
      <text class="app-slogan">高效外呼 · 智能管理</text>
    </view>

    <!-- 功能入口 -->
    <view class="menu-section">
      <view class="menu-item" @tap="openUserAgreement">
        <text class="menu-icon">📄</text>
        <text class="menu-label">用户服务协议</text>
        <text class="menu-arrow">›</text>
      </view>
      <view class="menu-item" @tap="openPrivacyPolicy">
        <text class="menu-icon">🔒</text>
        <text class="menu-label">隐私政策</text>
        <text class="menu-arrow">›</text>
      </view>
      <view class="menu-item" @tap="showFeedback">
        <text class="menu-icon">💬</text>
        <text class="menu-label">意见反馈</text>
        <text class="menu-arrow">›</text>
      </view>
      <view class="menu-item" @tap="showContact">
        <text class="menu-icon">📞</text>
        <text class="menu-label">联系我们</text>
        <text class="menu-arrow">›</text>
      </view>
    </view>

    <!-- 公司信息 -->
    <view class="company-section">
      <text class="company-name">广州仙狐网络科技有限公司</text>
      <text class="copyright">Copyright © 2024-2026 All Rights Reserved</text>
    </view>
  </view>
</template>

<script setup lang="ts">
import { APP_VERSION } from '@/config/app'

const appVersion = APP_VERSION

// 打开用户协议
const openUserAgreement = () => {
  uni.navigateTo({ url: '/pages/agreement/user-agreement' })
}
```

```
// 打开隐私政策
const openPrivacyPolicy = () => {
  uni.navigateTo({ url: '/pages/agreement/privacy-policy' })
}

// 意见反馈
const showFeedback = () => {
  uni.showModal({
    title: '意见反馈',
    content: '如有问题或建议，请通过以下方式联系我们：\n\n📞 客服电话：13570727234\n📧 邮
    showCancel: false,
    confirmText: '我知道了'
  })
}

// 联系我们
const showContact = () => {
  uni.showActionSheet({
    itemList: ['拨打客服电话', '复制客服微信', '复制邮箱地址'],
    success: (res) => {
      if (res.tapIndex === 0) {
        uni.makePhoneCall({
          phoneNumber: '13570727234',
          fail: () => {
            uni.setClipboardData({
              data: '13570727234',
              success: () => {
                uni.showToast({ title: '电话已复制', icon: 'success' })
              }
            })
          }
        })
      } else if (res.tapIndex === 1) {
        uni.setClipboardData({
          data: 'nxys789',
          success: () => {
            uni.showToast({ title: '微信号已复制', icon: 'success' })
          }
        })
      } else if (res.tapIndex === 2) {
        uni.setClipboardData({
          data: 'xianhuquwang@163.com',
          success: () => {
            uni.showToast({ title: '邮箱已复制', icon: 'success' })
          }
        })
      }
    }
  })
```

```
}
</script>

<style lang="scss" scoped>
.about-page {
  min-height: 100vh;
  background: #f5f5f5;
  padding-bottom: 60rpx;

  .app-info {
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 80rpx 32rpx 60rpx;
    background: linear-gradient(135deg, #6EE7B7 0%, #34D399 100%);

    .app-logo {
      width: 160rpx;
      height: 160rpx;
      background: #fff;
      border-radius: 32rpx;
      padding: 20rpx;
      box-shadow: 0 8rpx 32rpx rgba(0, 0, 0, 0.15);
    }

    .app-name {
      font-size: 44rpx;
      font-weight: bold;
      color: #fff;
      margin-top: 32rpx;
    }

    .app-version {
      font-size: 28rpx;
      color: rgba(255, 255, 255, 0.9);
      margin-top: 12rpx;
    }

    .app-slogan {
      font-size: 26rpx;
      color: rgba(255, 255, 255, 0.8);
      margin-top: 16rpx;
    }
  }

  .menu-section {
    background: #fff;
    margin: 32rpx;
    border-radius: 16rpx;
    overflow: hidden;
```

```
  .menu-item {
    display: flex;
    align-items: center;
    padding: 32rpx;
    border-bottom: 1rpx solid #F3F4F6;

    &:last-child {
      border-bottom: none;
    }

    &:active {
      background: #F9FAFB;
    }

    .menu-icon {
      font-size: 36rpx;
      margin-right: 24rpx;
    }

    .menu-label {
      flex: 1;
      font-size: 30rpx;
      color: #1F2937;
    }

    .menu-arrow {
      font-size: 32rpx;
      color: #D1D5DB;
    }
  }
}

.company-section {
  text-align: center;
  padding: 40rpx 32rpx;

  .company-name {
    display: block;
    font-size: 26rpx;
    color: #6B7280;
    margin-bottom: 12rpx;
  }

  .copyright {
    display: block;
    font-size: 22rpx;
    color: #9CA3AF;
  }
}
```

```
}
</style>


// -------- file: src/pages/agreement/user-agreement.vue --------
<template>
  <view class="agreement-page">
    <scroll-view scroll-y class="scroll-content">
      <view class="content">
        <text class="title">用户服务协议</text>
        <text class="update-time">更新日期：2026年1月2日</text>
        <text class="update-time">生效日期：2026年1月2日</text>

        <!-- 引言 -->
        <view class="section">
          <text class="section-content intro">
            欢迎您使用CRM外呼助手（以下简称"本应用"）！本应用由广州仙狐网络科技有限公司（
          </text>
          <text class="section-content intro">
            【特别提示】当您点击"同意"按钮或以其他方式确认接受本协议，或您下载、安装、注册
          </text>
        </view>

        <!-- 第一条 -->
        <view class="section">
          <text class="section-title">第一条 协议的接受与修改</text>

          <view class="subsection">
            <text class="subsection-title">1.1 协议的接受</text>
            <text class="section-content">
              本协议是您与广州仙狐网络科技有限公司之间关于使用本应用服务所订立的协议。本协
            </text>
          </view>

          <view class="subsection">
            <text class="subsection-title">1.2 协议的修改</text>
            <text class="section-content">
              我们有权根据国家法律法规的变化、业务发展需要等原因，对本协议进行修改。修改后
            </text>
          </view>
        </view>

        <!-- 第二条 -->
        <view class="section">
          <text class="section-title">第二条 服务内容</text>

          <view class="subsection">
            <text class="subsection-title">2.1 服务概述</text>
            <text class="section-content">
              本应用是一款企业级客户关系管理（CRM）移动办公工具，旨在帮助企业提高外呼效率
```

```
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">2.2 主要功能</text>
    <view class="list-content">
      <text class="list-item">（1）外呼管理：支持通过工作手机发起外呼，自动记录通
      <text class="list-item">（2）客户管理：查看和管理客户信息，记录客户跟进情况
      <text class="list-item">（3）订单管理：查看订单状态，处理订单相关事务，跟踪
      <text class="list-item">（4）数据统计：查看个人业绩数据、通话统计、客户分析
      <text class="list-item">（5）消息通知：接收系统消息、业务提醒、任务通知等；
      <text class="list-item">（6）设备绑定：支持工作手机与PC端系统的绑定联动；</
      <text class="list-item">（7）其他企业办公相关功能。</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">2.3 使用限制</text>
    <text class="section-content">
      本应用仅供企业内部员工使用，需要通过企业管理员分配的账号登录使用。未经授权，
    </text>
  </view>
</view>

<!-- 第三条 -->
<view class="section">
  <text class="section-title">第三条 用户账号</text>

  <view class="subsection">
    <text class="subsection-title">3.1 账号获取</text>
    <text class="section-content">
      您的账号由企业管理员创建和分配。账号信息包括但不限于用户名、初始密码、所属部
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">3.2 账号安全</text>
    <view class="list-content">
      <text class="list-item">（1）您应妥善保管账号和密码，不得将账号密码告知他人
      <text class="list-item">（2）您应对账号下的所有行为负责，包括但不限于通话记
      <text class="list-item">（3）如发现账号被盗用或存在安全风险，应立即通知企业
      <text class="list-item">（4）因您自身原因导致账号密码泄露所造成的损失，由您
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">3.3 账号使用规范</text>
    <view class="list-content">
      <text class="list-item">（1）您不得将账号转让、出借、出租给他人使用；</text
      <text class="list-item">（2）您不得使用他人账号登录本应用；</text>
```

```
      <text class="list-item">（3）您不得通过任何方式获取他人账号信息。</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">3.4 账号注销</text>
    <text class="section-content">
      当您离职或不再需要使用本应用时，企业管理员将注销您的账号。账号注销后，您将无
    </text>
  </view>
</view>

<!-- 第四条 -->
<view class="section">
  <text class="section-title">第四条 用户行为规范</text>

  <view class="subsection">
    <text class="subsection-title">4.1 基本规范</text>
    <text class="section-content">
      您在使用本应用时，应遵守国家法律法规、企业规章制度以及本协议的各项规定。
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">4.2 禁止行为</text>
    <text class="section-content">您不得利用本应用从事以下行为：</text>
    <view class="list-content">
      <text class="list-item">（1）违反国家法律法规的行为，包括但不限于电话诈骗、
      <text class="list-item">（2）骚扰、欺诈客户，恶意拨打骚扰电话；</text>
      <text class="list-item">（3）泄露客户隐私信息、企业商业秘密；</text>
      <text class="list-item">（4）传播违法、有害、淫秽、暴力等不良信息；</text>
      <text class="list-item">（5）破坏、干扰本应用的正常运行；</text>
      <text class="list-item">（6）利用技术手段非法获取数据或攻击系统；</text>
      <text class="list-item">（7）其他可能损害我们、企业或第三方合法权益的行为。
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">4.3 违规处理</text>
    <text class="section-content">
      如您违反上述规范，我们有权采取以下措施：暂停或终止您使用本应用的权利；向企业
    </text>
  </view>
</view>

<!-- 第五条 -->
<view class="section">
  <text class="section-title">第五条 知识产权</text>

  <view class="subsection">
```

```
    <text class="subsection-title">5.1 应用知识产权</text>
    <text class="section-content">
      本应用的所有内容，包括但不限于文字、图片、音频、视频、软件、程序、界面设计、
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.2 使用限制</text>
    <text class="section-content">
      未经我们书面许可，您不得复制、修改、传播、出售、出租本应用的任何内容，不得对
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.3 业务数据归属</text>
    <text class="section-content">
      您在使用本应用过程中产生的业务数据（包括但不限于客户信息、订单数据、通话记录
    </text>
  </view>
</view>

<!-- 第六条 -->
<view class="section">
  <text class="section-title">第六条 免责声明</text>

  <view class="subsection">
    <text class="subsection-title">6.1 服务中断</text>
    <text class="section-content">
      因网络状况、通讯线路、系统维护升级等原因造成的服务中断或延迟，我们不承担责任
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">6.2 不可抗力</text>
    <text class="section-content">
      因不可抗力（如自然灾害、战争、政府行为、法律法规变化等）导致的服务中断或无法
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">6.3 用户原因</text>
    <text class="section-content">
      因您自身原因（如账号泄露、操作失误、设备故障等）造成的损失，我们不承担责任。
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">6.4 第三方服务</text>
    <text class="section-content">
      本应用可能包含第三方服务链接或接口，我们对第三方服务的内容、安全性和行为不承
```

```
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">6.5 通话内容</text>
    <text class="section-content">
      您使用本应用进行的通话内容和业务行为，由您自行负责。我们不对通话内容进行审核
    </text>
  </view>
</view>

<!-- 第七条 -->
<view class="section">
  <text class="section-title">第七条 协议终止</text>

  <view class="subsection">
    <text class="subsection-title">7.1 用户终止</text>
    <text class="section-content">
      您可以随时停止使用本应用。如需注销账号，请联系企业管理员。
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">7.2 我们终止</text>
    <text class="section-content">
      如您违反本协议或相关法律法规，我们有权立即终止向您提供服务，无需事先通知。
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">7.3 终止后果</text>
    <text class="section-content">
      协议终止后，您应停止使用本应用。我们有权删除您的账号和相关数据，但法律法规另
    </text>
  </view>
</view>

<!-- 第八条 -->
<view class="section">
  <text class="section-title">第八条 争议解决</text>

  <view class="subsection">
    <text class="subsection-title">8.1 适用法律</text>
    <text class="section-content">
      本协议的订立、执行、解释及争议解决均适用中华人民共和国法律（不包括港澳台地区
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">8.2 争议解决方式</text>
```

```
            <text class="section-content">
              因本协议引起的或与本协议有关的任何争议，双方应首先通过友好协商解决。协商不成
            </text>
          </view>
        </view>

        <!-- 第九条 -->
        <view class="section">
          <text class="section-title">第九条 其他条款</text>

          <view class="subsection">
            <text class="subsection-title">9.1 条款独立性</text>
            <text class="section-content">
              本协议的任何条款被认定为无效或不可执行，不影响其他条款的效力。
            </text>
          </view>

          <view class="subsection">
            <text class="subsection-title">9.2 完整协议</text>
            <text class="section-content">
              本协议构成您与我们之间关于使用本应用的完整协议，取代之前就同一事项达成的任何
            </text>
          </view>

          <view class="subsection">
            <text class="subsection-title">9.3 联系方式</text>
            <text class="section-content">
              如您对本协议有任何疑问，可通过以下方式联系我们：
            </text>
            <view class="contact-info">
              <text class="contact-item">公司名称：广州仙狐网络科技有限公司</text>
              <text class="contact-item">客服电话：13570727234</text>
              <text class="contact-item">电子邮箱：xianhuquwang@163.com</text>
              <text class="contact-item">微信客服：nxys789</text>
              <text class="contact-item">公司地址：广州市黄埔区南翔一路68号</text>
            </view>
          </view>
        </view>

        <!-- 结尾 -->
        <view class="footer-section">
          <text class="footer-text">广州仙狐网络科技有限公司</text>
          <text class="footer-text">2026年1月2日</text>
        </view>
      </view>
    </scroll-view>
  </view>
</template>

<script setup lang="ts">
```

```
</script>

<style lang="scss" scoped>
.agreement-page {
  height: 100vh;
  background: #fff;

  .scroll-content {
    height: 100%;
  }

  .content {
    padding: 32rpx;
    padding-bottom: 80rpx;

    .title {
      display: block;
      font-size: 44rpx;
      font-weight: bold;
      color: #1F2937;
      text-align: center;
      margin-bottom: 20rpx;
    }

    .update-time {
      display: block;
      font-size: 24rpx;
      color: #9CA3AF;
      text-align: center;
      margin-bottom: 8rpx;
    }

    .section {
      margin-top: 48rpx;
      margin-bottom: 32rpx;

      .section-title {
        display: block;
        font-size: 34rpx;
        font-weight: bold;
        color: #1F2937;
        margin-bottom: 24rpx;
        padding-bottom: 16rpx;
        border-bottom: 2rpx solid #E5E7EB;
      }

      .section-content {
        display: block;
        font-size: 28rpx;
        color: #4B5563;
```

```
    line-height: 1.9;
    text-align: justify;
    margin-bottom: 16rpx;

    &.intro {
      text-indent: 2em;
      margin-bottom: 20rpx;
    }
  }
}

.subsection {
  margin-bottom: 28rpx;
  padding-left: 16rpx;

  .subsection-title {
    display: block;
    font-size: 30rpx;
    font-weight: 600;
    color: #374151;
    margin-bottom: 12rpx;
  }
}

.list-content {
  padding-left: 8rpx;

  .list-item {
    display: block;
    font-size: 28rpx;
    color: #4B5563;
    line-height: 1.9;
    margin-bottom: 12rpx;
    padding-left: 16rpx;
  }
}

.contact-info {
  background: #F9FAFB;
  padding: 24rpx;
  border-radius: 12rpx;
  margin-top: 16rpx;

  .contact-item {
    display: block;
    font-size: 26rpx;
    color: #4B5563;
    line-height: 2;
  }
}
```

```
    .footer-section {
      margin-top: 60rpx;
      text-align: right;
      padding-right: 32rpx;

      .footer-text {
        display: block;
        font-size: 26rpx;
        color: #6B7280;
        line-height: 1.8;
      }
    }
  }
}
</style>


// -------- file: src/pages/agreement/privacy-policy.vue --------
<template>
  <view class="agreement-page">
    <scroll-view scroll-y class="scroll-content">
      <view class="content">
        <text class="title">隐私政策</text>
        <text class="update-time">更新日期：2026年1月2日</text>
        <text class="update-time">生效日期：2026年1月2日</text>

        <!-- 引言 -->
        <view class="section">
          <text class="section-content intro">
            广州仙狐网络科技有限公司（以下简称"我们"）非常重视用户的隐私保护。本《隐私政策
          </text>
          <text class="section-content intro">
            【特别提示】请您在使用CRM外呼助手（以下简称"本应用"）前，仔细阅读并充分理解本
          </text>
        </view>

        <!-- 第一条 -->
        <view class="section">
          <text class="section-title">第一条 我们收集的信息</text>
          <text class="section-content">
            为了向您提供服务，我们可能会收集以下类型的信息：
          </text>

          <view class="subsection">
            <text class="subsection-title">1.1 账号信息</text>
            <view class="list-content">
              <text class="list-item">• 用户名、密码（加密存储，我们无法获取明文密码）</te
              <text class="list-item">• 员工姓名、工号、所属部门</text>
              <text class="list-item">• 职位、角色权限信息</text>
```

```
      <text class="list-item">• 联系方式（由企业管理员录入）</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">1.2 设备信息</text>
    <view class="list-content">
      <text class="list-item">• 设备型号、品牌、操作系统版本</text>
      <text class="list-item">• 设备唯一标识符（用于设备绑定和安全验证）</text>
      <text class="list-item">• 网络状态信息（WiFi/移动网络）</text>
      <text class="list-item">• 应用版本信息</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">1.3 通话相关信息</text>
    <view class="list-content">
      <text class="list-item">• 通话记录（通话时间、时长、通话状态）</text>
      <text class="list-item">• 通话录音（需要您授权存储权限）</text>
      <text class="list-item">• 客户电话号码（用于外呼功能）</text>
      <text class="list-item">• 通话备注和跟进记录</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">1.4 日志信息</text>
    <view class="list-content">
      <text class="list-item">• 应用使用日志、操作记录</text>
      <text class="list-item">• 错误日志（用于问题排查和服务改进）</text>
      <text class="list-item">• 登录时间、登录IP地址</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">1.5 位置信息（可选）</text>
    <text class="section-content">
      仅在您明确授权后收集，用于考勤打卡、外勤签到等功能。您可以随时在系统设置中关
    </text>
  </view>
</view>

<!-- 第二条 -->
<view class="section">
  <text class="section-title">第二条 我们如何使用您的信息</text>
  <text class="section-content">
    我们收集的信息将用于以下目的：
  </text>

  <view class="subsection">
    <text class="subsection-title">2.1 提供核心服务</text>
```

```
    <view class="list-content">
      <text class="list-item">• 验证您的身份，完成登录认证</text>
      <text class="list-item">• 提供外呼、客户管理等业务功能</text>
      <text class="list-item">• 记录和管理通话信息</text>
      <text class="list-item">• 实现设备绑定和数据同步</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">2.2 保障服务安全</text>
    <view class="list-content">
      <text class="list-item">• 设备绑定和安全验证</text>
      <text class="list-item">• 防止账号被盗用或滥用</text>
      <text class="list-item">• 识别和防范安全风险</text>
      <text class="list-item">• 检测和防止欺诈行为</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">2.3 改进服务质量</text>
    <view class="list-content">
      <text class="list-item">• 分析应用使用情况，优化功能体验</text>
      <text class="list-item">• 排查和修复技术问题</text>
      <text class="list-item">• 开发新功能和服务</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">2.4 企业管理需要</text>
    <view class="list-content">
      <text class="list-item">• 向企业管理员提供员工工作数据统计</text>
      <text class="list-item">• 支持企业进行业务分析和决策</text>
      <text class="list-item">• 生成业绩报表和数据分析</text>
    </view>
  </view>
</view>

<!-- 第三条 -->
<view class="section">
  <text class="section-title">第三条 信息的存储与保护</text>

  <view class="subsection">
    <text class="subsection-title">3.1 存储位置</text>
    <text class="section-content">
      您的信息存储在企业指定的服务器上，服务器位于中华人民共和国境内。我们采用业务
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">3.2 存储期限</text>
```

```
    <view class="list-content">
      <text class="list-item">• 账号信息：在您使用服务期间持续保存，账号注销后按规
      <text class="list-item">• 通话记录：根据企业设置的保留期限保存，通常为1-3年，
      <text class="list-item">• 通话录音：根据企业设置的保留期限保存</text>
      <text class="list-item">• 日志信息：通常保存不超过6个月</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">3.3 安全措施</text>
    <view class="list-content">
      <text class="list-item">• 数据传输采用HTTPS/SSL加密</text>
      <text class="list-item">• 敏感信息（如密码）采用不可逆加密存储</text>
      <text class="list-item">• 实施严格的访问控制，限制数据访问权限</text>
      <text class="list-item">• 定期进行安全评估和漏洞修复</text>
      <text class="list-item">• 建立数据备份和灾难恢复机制</text>
    </view>
  </view>
</view>

<!-- 第四条 -->
<view class="section">
  <text class="section-title">第四条 信息的共享与披露</text>
  <text class="section-content">
    我们不会向第三方出售您的个人信息。在以下情况下，我们可能会共享您的信息：
  </text>

  <view class="subsection">
    <text class="subsection-title">4.1 企业内部共享</text>
    <view class="list-content">
      <text class="list-item">• 向您所在企业的管理员提供必要的工作数据</text>
      <text class="list-item">• 在企业内部进行业务协作时共享相关信息</text>
      <text class="list-item">• 生成团队业绩报表时使用汇总数据</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">4.2 法律要求</text>
    <view class="list-content">
      <text class="list-item">• 根据法律法规的强制性要求</text>
      <text class="list-item">• 响应政府部门的合法要求</text>
      <text class="list-item">• 为保护我们、用户或公众的合法权益</text>
      <text class="list-item">• 配合司法机关的调查取证</text>
    </view>
  </view>

  <view class="subsection">
    <text class="subsection-title">4.3 第三方服务</text>
    <text class="section-content">
      使用第三方云服务存储数据时，我们会确保其具备相应的安全保障能力，并签署数据保
```

```
    </text>
  </view>
</view>

<!-- 第五条 -->
<view class="section">
  <text class="section-title">第五条 您的权利</text>
  <text class="section-content">
    根据相关法律法规，您对您的个人信息享有以下权利：
  </text>

  <view class="subsection">
    <text class="subsection-title">5.1 访问权</text>
    <text class="section-content">
      您可以查看您的账号信息、通话记录等个人数据。如需获取更多信息，可联系企业管理
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.2 更正权</text>
    <text class="section-content">
      如发现您的个人信息有误，可联系企业管理员进行更正。
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.3 删除权</text>
    <text class="section-content">
      您可以请求删除您的个人信息，但可能受到法律法规和企业规定的限制。某些信息可能
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.4 撤回同意</text>
    <text class="section-content">
      您可以随时撤回对本隐私政策的同意，但这可能导致您无法继续使用本应用的部分或全
    </text>
  </view>

  <view class="subsection">
    <text class="subsection-title">5.5 注销账号</text>
    <text class="section-content">
      当您离职或不再需要使用本应用时，可联系企业管理员注销账号。账号注销后，我们将
    </text>
  </view>
</view>

<!-- 第六条 -->
<view class="section">
  <text class="section-title">第六条 权限申请说明</text>
```

```
  <text class="section-content">
    本应用可能需要申请以下系统权限，我们承诺仅在必要时使用这些权限：
  </text>

  <view class="permission-table">
    <view class="permission-item">
      <text class="permission-name">📞 电话权限</text>
      <text class="permission-desc">用途：发起外呼、获取通话状态</text>
      <text class="permission-required">必要性：核心功能必需</text>
    </view>

    <view class="permission-item">
      <text class="permission-name">🎙 麦克风权限</text>
      <text class="permission-desc">用途：通话录音功能</text>
      <text class="permission-required">必要性：录音功能必需，可选择不授权</text>
    </view>

    <view class="permission-item">
      <text class="permission-name">📁 存储权限</text>
      <text class="permission-desc">用途：保存录音文件、缓存数据</text>
      <text class="permission-required">必要性：部分功能必需</text>
    </view>

    <view class="permission-item">
      <text class="permission-name">🌐 网络权限</text>
      <text class="permission-desc">用途：与服务器通信、同步数据</text>
      <text class="permission-required">必要性：核心功能必需</text>
    </view>

    <view class="permission-item">
      <text class="permission-name">🔔 通知权限</text>
      <text class="permission-desc">用途：接收消息推送、来电提醒</text>
      <text class="permission-required">必要性：消息功能必需，可选择不授权</text>
    </view>

    <view class="permission-item">
      <text class="permission-name">📷 相机权限</text>
      <text class="permission-desc">用途：扫描二维码绑定设备</text>
      <text class="permission-required">必要性：扫码功能必需，可选择不授权</text>
    </view>
  </view>

  <text class="section-content">
    您可以在手机系统设置中随时管理这些权限。关闭某些权限可能导致相关功能无法正常使
  </text>
</view>

<!-- 第七条 -->
<view class="section">
  <text class="section-title">第七条 未成年人保护</text>
```

```
          <text class="section-content">
            本应用为企业办公应用，不面向未成年人提供服务。如果我们发现在未经监护人同意的情
          </text>
        </view>

        <!-- 第八条 -->
        <view class="section">
          <text class="section-title">第八条 隐私政策的更新</text>
          <text class="section-content">
            我们可能会根据法律法规变化、业务发展需要等原因更新本隐私政策。更新后的政策将在
          </text>
          <text class="section-content">
            建议您定期查阅本隐私政策，以了解我们如何保护您的信息。如您在政策更新后继续使用
          </text>
        </view>

        <!-- 第九条 -->
        <view class="section">
          <text class="section-title">第九条 联系我们</text>
          <text class="section-content">
            如果您对本隐私政策有任何疑问、意见或建议，或需要行使您的个人信息权利，可以通过
          </text>
          <view class="contact-info">
            <text class="contact-item">公司名称：广州仙狐网络科技有限公司</text>
            <text class="contact-item">客服电话：13570727234</text>
            <text class="contact-item">电子邮箱：xianhuquwang@163.com</text>
            <text class="contact-item">微信客服：nxys789</text>
            <text class="contact-item">公司地址：广州市黄埔区南翔一路68号</text>
          </view>
          <text class="section-content">
            我们将在收到您的反馈后15个工作日内回复。
          </text>
        </view>

        <!-- 结尾 -->
        <view class="footer-section">
          <text class="footer-text">广州仙狐网络科技有限公司</text>
          <text class="footer-text">2026年1月2日</text>
        </view>
      </view>
    </scroll-view>
  </view>
</template>

<script setup lang="ts">
</script>

<style lang="scss" scoped>
.agreement-page {
  height: 100vh;
```

```
background: #fff;

.scroll-content {
  height: 100%;
}

.content {
  padding: 32rpx;
  padding-bottom: 80rpx;

  .title {
    display: block;
    font-size: 44rpx;
    font-weight: bold;
    color: #1F2937;
    text-align: center;
    margin-bottom: 20rpx;
  }

  .update-time {
    display: block;
    font-size: 24rpx;
    color: #9CA3AF;
    text-align: center;
    margin-bottom: 8rpx;
  }

  .section {
    margin-top: 48rpx;
    margin-bottom: 32rpx;

    .section-title {
      display: block;
      font-size: 34rpx;
      font-weight: bold;
      color: #1F2937;
      margin-bottom: 24rpx;
      padding-bottom: 16rpx;
      border-bottom: 2rpx solid #E5E7EB;
    }

    .section-content {
      display: block;
      font-size: 28rpx;
      color: #4B5563;
      line-height: 1.9;
      text-align: justify;
      margin-bottom: 16rpx;

      &.intro {
```

```
      text-indent: 2em;
      margin-bottom: 20rpx;
    }
  }
}

.subsection {
  margin-bottom: 28rpx;
  padding-left: 16rpx;

  .subsection-title {
    display: block;
    font-size: 30rpx;
    font-weight: 600;
    color: #374151;
    margin-bottom: 12rpx;
  }
}

.list-content {
  padding-left: 8rpx;

  .list-item {
    display: block;
    font-size: 28rpx;
    color: #4B5563;
    line-height: 1.9;
    margin-bottom: 10rpx;
    padding-left: 16rpx;
  }
}

.permission-table {
  background: #F9FAFB;
  border-radius: 12rpx;
  padding: 16rpx;
  margin: 20rpx 0;

  .permission-item {
    padding: 20rpx;
    border-bottom: 1rpx solid #E5E7EB;

    &:last-child {
      border-bottom: none;
    }

    .permission-name {
      display: block;
      font-size: 28rpx;
      font-weight: 600;
```

```
          color: #1F2937;
          margin-bottom: 8rpx;
        }

        .permission-desc {
          display: block;
          font-size: 26rpx;
          color: #4B5563;
          margin-bottom: 4rpx;
        }

        .permission-required {
          display: block;
          font-size: 24rpx;
          color: #6B7280;
        }
      }
    }

    .contact-info {
      background: #F9FAFB;
      padding: 24rpx;
      border-radius: 12rpx;
      margin: 16rpx 0;

      .contact-item {
        display: block;
        font-size: 26rpx;
        color: #4B5563;
        line-height: 2;
      }
    }

    .footer-section {
      margin-top: 60rpx;
      text-align: right;
      padding-right: 32rpx;

      .footer-text {
        display: block;
        font-size: 26rpx;
        color: #6B7280;
        line-height: 1.8;
      }
    }
  }
}
</style>
```